

Submission Worksheet

Submission Data

Course: IT202-010-S2026

Assignment: IT202 Milestone 2

Student: Pratik P. (pbp5)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 5/4/2026 4:40:20 PM

Updated: 5/4/2026 8:24:43 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-2/grading/pbp5>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-2/view/pbp5>

Instructions

- Overview Link: https://youtu.be/t6X8dzZt_a0
- 1. Refer to Milestone2 of this doc:
<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
- 2. Ensure you read all instructions and objectives before starting.
- 3. Ensure you've gone through each lesson related to this Milestone
- 4. Switch to the Milestone2 branch
 - 1. `git checkout Milestone2` (ensure proper starting branch)
 - 2. `git pull origin Milestone2` (ensure history is up to date)
- 5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
- 6. Once finished, click "Submit and Export"
- 7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`
 - 3. `git push origin Milestone2`
 - 4. On Github merge the pull request from Milestone2 to qa
 - 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the Render.com prod links work)
- 8. Upload the same PDF to Canvas
- 9. Sync Local
 - 1. `git checkout qa`
 - 2. `git pull origin qa`

Section #1: (2 pts.) Data

Progress: 100%

≡ Task #1 (0 50 pts) - API Data Table

Part 1:

Progress: 100%

Details:

- Show the table(s) structure of the table made to hold your API
- There should be at least 3 properties/fields beyond id, created, modified, and respective is_api or api_id columns
- Logical types and constraints should be set
- Show at least a few records from each API table (samples should include both API fetched data and custom entries noted by an api_id or is_api column)

```
1 CREATE TABLE IF NOT EXISTS "cities" (  
2   "id" int NOT NULL AUTO_INCREMENT,  
3   "name" varchar(255) NOT NULL,  
4   "latitude" decimal(10,6),  
5   "longitude" decimal(10,6),  
6   "population" int,  
7   "country_code" varchar(3),  
8   "api_id" varchar(100) UNIQUE,  
9   "created" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
10  "modified" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
11  "is_api" tinyint(1) DEFAULT '0',  
12  "is_deleted" tinyint(1) DEFAULT '0',  
13  PRIMARY KEY ("id")  
14 );
```

create cities table

```
1 CREATE TABLE IF NOT EXISTS "countries" (  
2   "id" int NOT NULL AUTO_INCREMENT,  
3   "name" varchar(255) NOT NULL,  
4   "code" varchar(10) NOT NULL UNIQUE,  
5   "currency" varchar(10),  
6   "api_id" varchar(100),  
7   "created" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
8   "modified" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
9   "is_api" tinyint(1) DEFAULT '0',  
10  PRIMARY KEY ("id")  
11 );
```

create country table

SELECT * FROM cities LIMIT 100

	id	name	latitude	longitude	population	country_code	api_id	created	modified	is_api	is_deleted
	int	varchar(255)	decimal(10,6)	decimal(10,6)	int	varchar(3)	varchar(100)	timestamp	timestamp	tinyint(1)	tinyint(1)
>	1	New York City	40.714270	-74.005970	8500001	US	(NULL)	2025-05-02 22:18:21	2025-05-03 00:56:14	0	0
>	4	Chicago	41.881944	-87.627778	2746388	US	120473	2025-05-02 22:36:13	2025-05-02 22:36:13	1	0
>	5	Denver	39.739167	-104.984722	715522	US	126880	2025-05-03 21:00:07	2025-05-03 21:00:07	1	0
>	6	London	51.520200	-0.100020	8866180	GB	3602356	2025-05-03 21:14:13	2025-05-03 21:27:02	1	0

sql cities

SELECT * FROM countries LIMIT 100

	id	name	code	currency	api_id	created	modified	is_api
	int	varchar(255)	varchar(10)	varchar(10)	varchar(100)	timestamp	timestamp	tinyint(1)

	Q	int	varchar(255)	varchar(10)	varchar(10)	varchar(100)	timestamp	timestamp	tinyint(1)
	>	4	Canada	CA	CAD	(NULL)	2026-05-03 00:22:51	2026-05-03 00:22:51	0
	>	7	Mexico	MX	MXN	Q96	2026-05-03 00:29:56	2026-05-03 00:29:56	1
	>	8	Azerbaijan	AZD	AZD	Q227	2026-05-03 21:17:31	2026-05-04 19:08:57	1

sql countries

 Saved: 5/4/2026 4:51:30 PM

Part 2:

Progress: 100%

Details:

- Add a direct link to your sql file from Github (should end in .sql)

URL #1

https://github.com/pbp965/pbp5-IT20260101Milestone2/public_html/project/sql/006_create_table_cities.sql



URL

https://github.com/pbp965/pbp5-IT20260101Milestone2/public_html/project/sql/006_create_table_cities.sql



URL #2

https://github.com/pbp965/pbp5-IT20260101Milestone2/public_html/project/sql/007_create_table_countries.sql



URL

https://github.com/pbp965/pbp5-IT20260101Milestone2/public_html/project/sql/007_create_table_countries.sql



 Saved: 5/4/2026 4:51:30 PM

Part 3:

Progress: 100%

Details:

- Briefly note what each field/property represents

Your Response:

cities: id: id of entry, int name: name of city, varchar latitude: float of the latitude of the city
 longitutde: float of the longitude of the city population: population of the city, int api_id: the id of the city within the api, int country_code: country code of the country the city is located in varchar
 created: date created modified: date modified, if not modified date is set to the same as the date in created is_api: determines wether city if fetched from api and not user created, int, 1 = from api, 0 = user created

countries: id: id of entry, int name: name of country, varchar population: population of the country, int currency: currency code of country, varchar api_id: the id of the country within the api, int
 created: date created modified: date modified, if not modified date is set to the same as the date in created is_api: determines wether city if fetched from api and not user created, int, 1 = from api, 0 = user created

Task #2 (0.50 pts.) - Data Transformation

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles the data transformation/conversion (from the API)
- Show a Render.com qa example of API getting fetched and loaded into the DB

```
$transformedResult = [];
// transform data to match our DB structure
if (isset($result["data"])) {
    foreach ($result["data"] as $city) {
        $transformedResult[] = [
            "api id" => $city["id"] ?? null,
            "name" => $city["name"] ?? "",
            "latitude" => isset($city["latitude"]) ? floatval($city["latitude"]) : null,
            "longitude" => isset($city["longitude"]) ? floatval($city["longitude"]) : null,
            "population" => isset($city["population"]) ? intval($city["population"]) : 0,
            "country_code" => $city["countryCode"] ?? null,
            "is_api" => 1
        ];
    }
}
return $transformedResult;
function fetch_city($namePrefix)
```

cities transformation

```
// transform data
$transformed = [];
if (isset($result["data"])) {
    foreach ($result["data"] as $country) {
        $transformed[] = [
            "api_id" => $country["wikiDataId"] ?? null,
            "name" => $country["name"] ?? "",
            "code" => $country["code"] ?? "",
            "currency" => $country["currencyCodes"][0] ?? null,
            "is_api" => 1
        ];
    }
}
return $transformed;
```

countries transformation

Render.com QA Example: Create or Fetch City

Preview:

```
[
  {
    "api id" => 337993,
    "name" => "Hillesholm",
    "latitude" => 56.08577777,
    "longitude" => 12.55611111,
    "population" => 1000000,
    "country_code" => "DK",
    "is_api" => 1
  }
]
```

fetch city

Render.com QA Example: Create or Fetch Countries

Preview:

```
[
  {
    "api id" => 337993,
    "name" => "Hillesholm",
    "latitude" => 56.08577777,
    "longitude" => 12.55611111,
    "population" => 1000000,
    "country_code" => "DK",
    "is_api" => 1
  }
]
```

```
Preview
array
(
    [0] => array
        (
            [lat] => 34.0
            [long] => -118.2
            [code1] => 33
            [country] => USA
            [city] => LA
        )
)
```

fetch country



Saved: 5/4/2026 8:13:05 PM

Part 2:

Progress: 100%

Details:

- Include Render.com prod link to API fetch page
- Include pull request link for this feature

URL #1

https://pbp5-it202-010-prod.onrender.com/project/admin/create_country.php



URL
https://pbp5-it202-010-prod.onrender.com/project/admin/create_country.php



URL #2

<https://github.com/pbp965/pbp5-it202-010-43>



URL
<https://github.com/pbp965/pbp5-it202-010-43>



URL #3

https://pbp5-it202-010-prod.onrender.com/project/admin/create_city.php#



URL
https://pbp5-it202-010-prod.onrender.com/project/admin/create_city.php#



Saved: 5/4/2026 8:13:05 PM

Part 3:

Progress: 100%

Details:

- Will you be using all the data or just a subset?
- Do you need to convert any data or extract pieces of a property's value?
- Briefly explain the transformation/extraction logic to get the API data to the shape your application intends to use

Your Response:

All the data will be used, longitude and latitude are converted into floats, population is converted into int, and the currency code from countries uses the first element as the code comes in an array.



Saved: 5/4/2026 8:13:05 PM

Task #3 (0.50 pts.) - API Duplicates

Progress: 100%

Details:

- Consider how duplicate entries are handled
 - What happens if you get the same result from the API for an entity that exists but has been unmodified?
 - What happens if you get the same result from the API for an entity that exists but has been modified?

Your Response:

The error will be caught and a message saying "This city already exists in the database" is flashed onto the screen.



Saved: 5/4/2026 5:09:20 PM

Task #4 (0.50 pts.) - API Fetching

Progress: 100%

Details:

- Consider how your application will trigger this data fetch
 - (i.e., periodic via cron job, poor man's cron job, on app start/awake, etc)
 - (i.e., User or Admin triggered via a specific page/action)
 - (i.e., Lazy loaded during searches for stale/missing content)

Your Response:

Admin triggered fetch with lazy loading



Saved: 5/4/2026 5:22:13 PM

Section #2: (1.5 pts.) Data Creation

Progress: 100%

Task #1 (0.60 pts.) - Validations

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the html validations (code)

- Show the php validations (code)
- Show the php validations (code)

```
//div id="create" style="display:none;" class="tab-target">
<form method="POST" onsubmit="return validatecreate();"
<div class="mb-2">
<label>City Name</label>
<input type="text" name="name" required>
</div>
<div class="mb-2">
<label>Latitude</label>
<input type="number" step="0.000001" name="latitude">
</div>
<div class="mb-2">
<label>Longitude</label>
<input type="number" step="0.000001" name="longitude">
</div>
<div class="mb-2">
<label>Population</label>
<input type="number" name="population">
</div>
<div class="mb-2">
<label>Country Code</label>
<input type="text" name="country_code" maxlength="5">
</div>
<input type="hidden" name="action" value="create">
<input type="submit" value="Create City" class="btn btn-primary">
</form>
</div>
```

city html

```
<div id="create" style="display:none;" class="tab-target">
<form method="POST" onsubmit="return validatecreate();"
<div class="mb-2">
<label>Country Name</label>
<input type="text" name="name" required>
</div>
<div class="mb-2">
<label>Country Code</label>
<input type="text" name="code" maxlength="5" required>
</div>
<div class="mb-2">
<label>Currency</label>
<input type="text" name="currency" maxlength="5">
</div>
<input type="hidden" name="action" value="create">
<input type="submit" value="Create Country" class="btn btn-primary">
</form>
</div>
```

country html

```
function validateFetch() {
let name = document.getElementById("namePrefix").value.trim();
if (!name) {
alert("City name is required");
return false;
}
return true;
}

function validateCreate() {
let name = document.querySelector("[name='name']").value.trim();
if (!name) {
alert("City name is required");
return false;
}
return true;
}
```

city js

```
function validateFetch() {
let name = document.querySelector("[name='namePrefix']").value.trim();
if (!name) {
alert("Country name is required");
return false;
}
return true;
}

function validateCreate() {
let name = document.querySelector("[name='name']").value.trim();
let code = document.querySelector("[name='code']").value.trim();

if (!name || !code) {
alert("Name and code are required");
return false;
}
return true;
}
```

country js

```
if (typeof validateFetch === 'function') {
// Function already exists
} else {
// Function does not exist
function validateFetch() {
// Function implementation
}
}

if (typeof validateCreate === 'function') {
// Function already exists
} else {
// Function does not exist
function validateCreate() {
// Function implementation
}
}
```

city php 1

```

    # Create a new instance of the class
    obj = cls(*args, **kwargs)

    # Call the __init__ method
    obj.__init__(*args, **kwargs)

    # Return the instance
    return obj

# Example usage
obj = MyClass(1, 2, 3)
obj.__init__(1, 2, 3)
print(obj.x, obj.y, obj.z)

```

city php 2

country.php



Saved: 5/4/2026 5:42:37 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the html validations used
- Briefly explain the javascript validation logic
- Briefly explain the php validation logic
- Note which role(s) have access to this create form

Your Response:

checkhtml: checks the type of the input (number or text), the length of a number (for numbers), and max length of the entry (for text if needed) **js:** only checks if a name is entered and checks if a country code is entered for countries **php:** checks whether the inputs are empty, duplicate, or if an error occurred only admins are allowed to insert a created city into the db



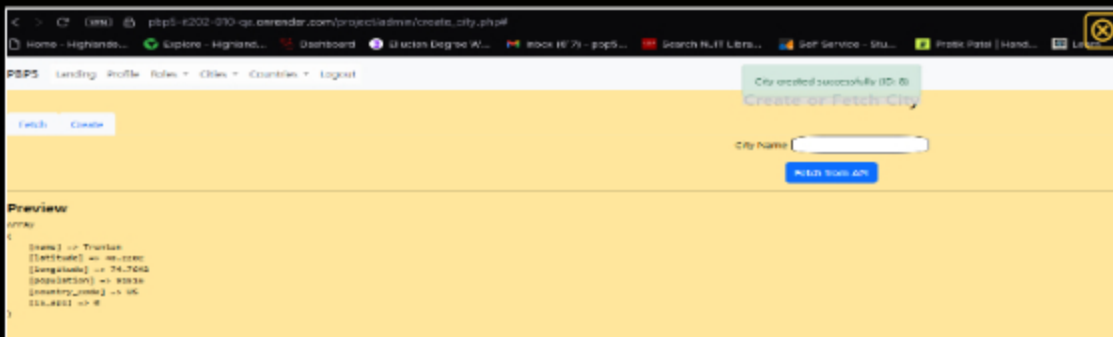
Saved: 5/4/2026 5:42:37 PM

Task #2 (0.60 pts.) - Examples

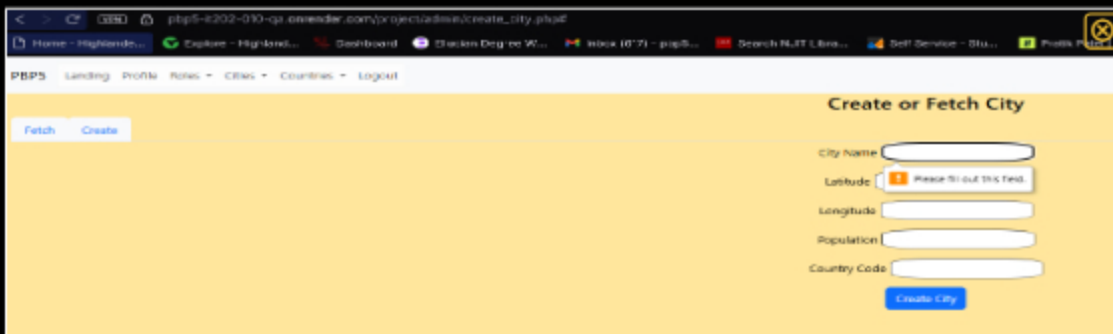
Progress: 100%

Details:

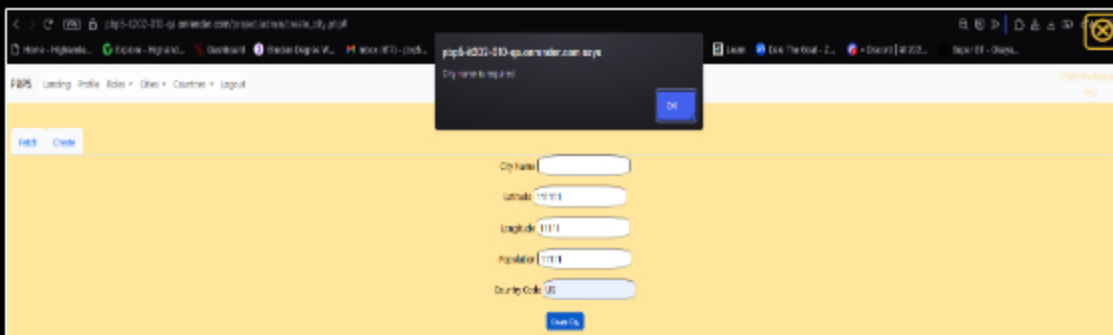
- Show a successful creation
- Show how a duplicate submission is handled (i.e., what message or behavior occurs)
- Demonstrate various validation errors
- Ensure Render.com qa url is visible



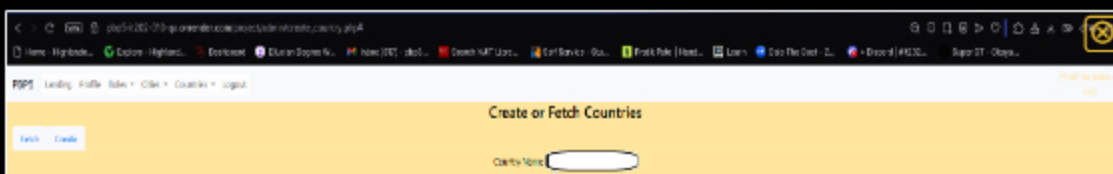
city created



city html



city js



Country Code:

html country 1

pbp5-it202-010-prod.onrender.com/project/admin/create_country.php

Home - Render.com - Explore - Hosted - Dockerize - Elastic Deploy W... - Video dIT - pop...

PEPS Landing Profile Roles Cities Countries Logout

Create or Fetch Countries

Fetch Create

Country Name:

Country Code:

Currency:

html country 2

pbp5-it202-010-prod.onrender.com/project/admin/create_country.php

Home - Render.com - Explore - Hosted - Dockerize - Elastic Deploy W... - Video dIT - pop...

PEPS Landing Profile Roles Cities Countries Logout

Create or Fetch Countries

Fetch Create

Country Name:

Country Code:

Currency:

pbp5-it202-010-prod.onrender.com says
Name and code are required

country.js



Saved: 5/4/2026 6:21:17 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://pbp5-it202-010-prod.onrender.com/project/admin/create_country.php#



URL

<https://pbp5-it202-010-prod.c>



URL #2

<https://github.com/pbp965/pbp5-IT202-pull104>



URL

<https://github.com/pbp965/p>



URL #3

<https://pbp5-it202-010->



URL

<https://pbp5-it202-010-prod.c>



Section #3: (1.5 pts.) Data List Page (Many Items)

Progress: 100%

≡ Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles the filter/sort logic
- Show the code that enforces the record limit (valid range 1-100, default 10 if out of range)
- Show the code that generates the list/grid output
- Styling must be applied

```
$allowed_sort = ["name", "population", "created"];
if (in_array($sort, $allowed_sort)) {
    $order = "created";
}

$orderby = str_replace($orderby, "ASC" + "DESC" + "DESC";

$query = "SELECT id, name, latitude, longitude, population, country_code, is_api
FROM cities
WHERE is_deleted = 0";

$params = [];

if ($name) {
    $query .= " AND name LIKE :name";
    $params[":name"] = "%$name%";
}

if ($country) {
    $query .= " AND country_code = :country";
    $params[":country"] = $country;
}

$query .= " ORDER BY $sort $order LIMIT :limit";
```

sort cities

```
if ($limit < 1 || $limit > 100) {  
    $limit = 10;  
}
```

limit cities/countries

```
<!--only?>
<?php foreach ($results as $row): ?>
    <tr>
        <td><?php echo($row["id"]); ?></td>
        <td><?php echo($row["name"]); ?></td>
        <td><?php echo($row["population"], "N/A"); ?></td>
        <td><?php echo($row["country_code"], "N/A"); ?></td>
        <td><?php echo $row["is_api"] ? "API": "Manual"; ?></td>
    </tr>
    <a href="<?php echo get url('admin/edit city.php'); ?>?id=<?php echo($row["id"]); ?>>edit/a> |
    <a href="<?php echo get url('admin/view city.php'); ?>?id=<?php echo($row["id"]); ?>>view/a> |
    <form method='post' style='display:inline;' onSubmit='return confirm("delete this city?");'?>
        <input type='hidden' name='id' value='<?php echo($row["id"]); ?>'>
        <input type='hidden' name='url' value='<?php echo get url('city.php'); ?>'>
    </form>
    </td>
    </tr>
</tr>
```

 Saved: 5/4/2026 6:56:27 PM

Details:

- Show a few variations of filter/sort applied (including no records - should display a "no results" message)
- Show both API-fetched and manually created entities present in the list
- Show the limit field in use with at least two different values
- Each list item should be a summary
- Each list item should have the following links
 - A link to a single view of the specific entity (i.e., a details page)
 - A link to delete this entity (this may be an admin-only functionality, but it should be present for the respective role)
 - A link to edit this entity (this may be an admin-only functionality, but it should be present for the respective role)
- Ensure Render.com qa url is visible

City List

City Name
Country Code
City Code
City ID

ID	Name	Population	Country	Source	Actions
6	Toronto	2,731,571	CA	Manual	View Edit Delete
7	Philadelphia	1,553,297	US	API	View Edit Delete
5	London	8,541,000	GB	API	View Edit Delete
5	Denver	715,502	US	API	View Edit Delete
6	Chicago	2,746,388	US	API	View Edit Delete
1	New York City	8,400,000	US	Manual	View Edit Delete

default city

City List

City Name
Country Code
City Code
City ID

ID	Name	Population	Country	Source	Actions
6	Toronto	2,731,571	US	Manual	View Edit Delete
6	Denver	715,502	US	API	View Edit Delete
7	Philadelphia	1,553,297	US	API	View Edit Delete
4	Chicago	2,746,388	US	API	View Edit Delete
1	New York City	8,400,000	US	Manual	View Edit Delete
6	London	8,541,000	GB	API	View Edit Delete

pop asc city

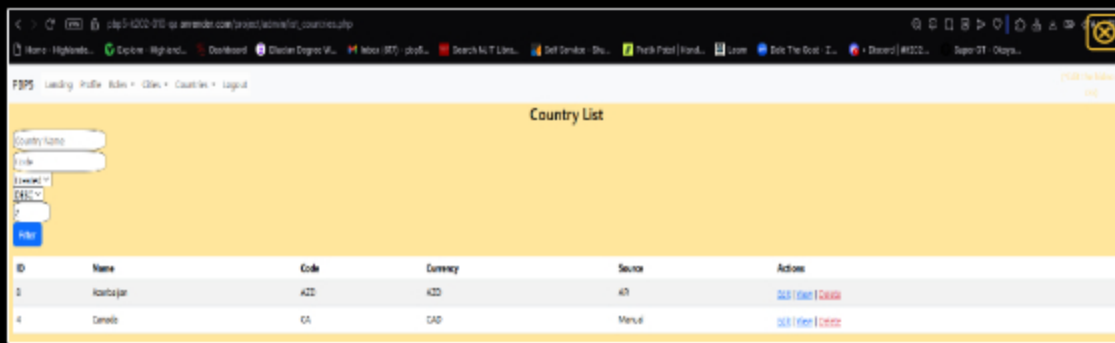
Country List

Country Name
Code
Currency
Country ID

ID	Name	Code	Currency	Source	Actions
7	Mexico	MX	MXN	API	View Edit Delete
4	Idj	ID	IDR	API	View Edit Delete
4	Canada	CA	CAD	Manual	View Edit Delete
5	Netherlands	AD	ADP	API	View Edit Delete

default country

default country



ID	Name	Code	Currency	Source	Actions
3	India	IN	INR	API	Add Edit Delete
4	Canada	CA	CAD	Manual	Add Edit Delete

code asc filter 2 country



Saved: 5/4/2026 6:57:49 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://pbb5-it202-010-prod.onrender.com/project/admin/list_cities.php?name=&country_code=&sort=population&order=asc&limit=10



URL

<https://pbb5-it202-010-prod.c>



URL #2

https://pbb5-it202-010-prod.onrender.com/project/admin/list_countries.php



URL

<https://pbb5-it202-010-prod.c>



URL #3

<https://github.com/pbb965/pbb5-IT202-010/pull/43>



URL

<https://github.com/pbb965/p>



Saved: 5/4/2026 8:13:22 PM

Section #4: (1.5 pts.) View Details Page (Single Item)

Progress: 100%

Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect back to the previous page with an appropriate message)
- Show the code that generates the entity output
- Styling must be applied

```

$id = $_GET["id"] - 1;
}

if ($id < 1) {
    flash("Invalid or missing city ID", "danger");
    die(header("Location: " . get_uri("admin/list_cities.php")));
}

$db = getDB();
$idCity = $id;

try {
    $stmt = $db->prepare("SELECT id, name, latitude, longitude, population, country_code, is_api, created
                        FROM cities
                        WHERE id = :id LIMIT 1");
    $stmt->execute(array(":id" => $id));
    $city = $stmt->fetch();

    if (!$city) {
        flash("City not found", "warning");
        die(header("Location: " . get_uri("admin/list_cities.php")));
    }
    catch (PDOException $e) {
        error_log("Error fetching city: " . var_export($e, true));
        flash("Error loading city", "danger");
        die(header("Location: " . get_uri("admin/list_cities.php")));
    }
} catch (PDOException $e) {
}

```

[view city](#)

output city

```

$id = $_GET["id", -1, false];

if ($id < 1) {
    flash("Invalid or missing country ID", "danger");
    die(header("Location: " . get_url("admin/list_countries.php")));
}

$id = addslashes($id);
$query = "SELECT * FROM countries WHERE id = $id LIMIT 1";
$stmt = $db->prepare($query);
$stmt->execute();
$result = $stmt->fetch();

if ($result) {
    flash("Country not found", "warning");
    die(header("Location: " . get_url("admin/list_countries.php")));
} catch (PDOException $e) {
    flash("Error fetching country: " . $e->getMessage(), "danger");
    die(header("Location: " . get_url("admin/list_countries.php")));
}

```

[view country](#)[illegible]

output country



Saved: 5/4/2026 7:32:02 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the entity output is generated
- Briefly note the styling choices

Your Response:

Uses an sql query using the city id to identify cities/countries, if id is less than 1, missing id message is flashed, put into \$city/\$country Respective elements are outputted from the \$city/\$country



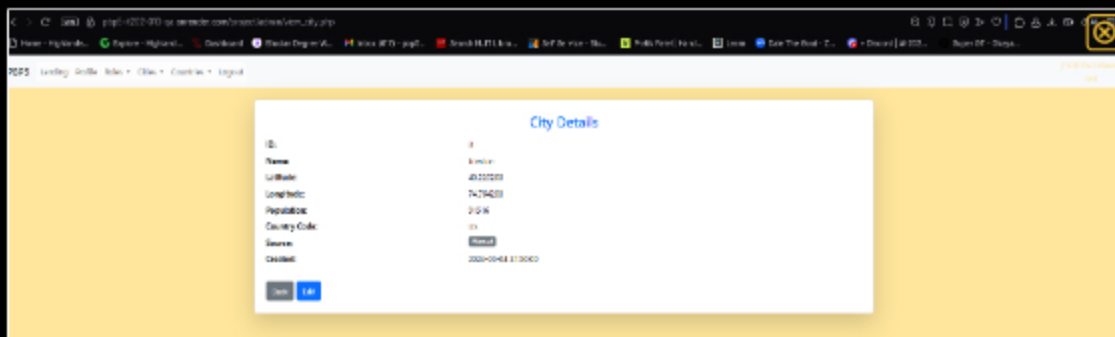
Saved: 5/4/2026 7:32:02 PM

Task #2 (0.60 pts.) - Examples

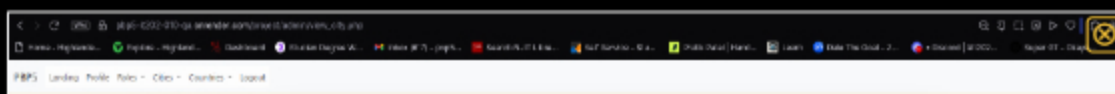
Progress: 100%

Details:

- Show a few examples of different entities
- Each entity item should have more details than the summary view (if possible)
- Each entity should have the following links
 - A link to edit the single entity (this may be an admin-only thing, but it should be present for the respective role)
 - A link to delete the single entity (this may be an admin-only thing, but it should be present for the respective role)

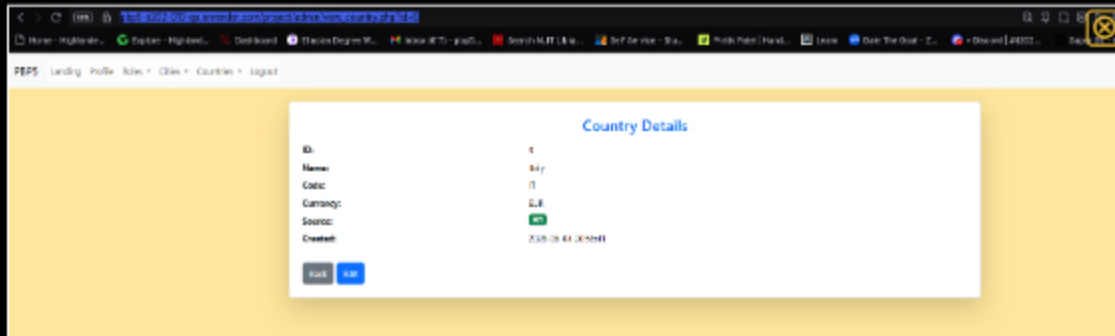


ex 1 city

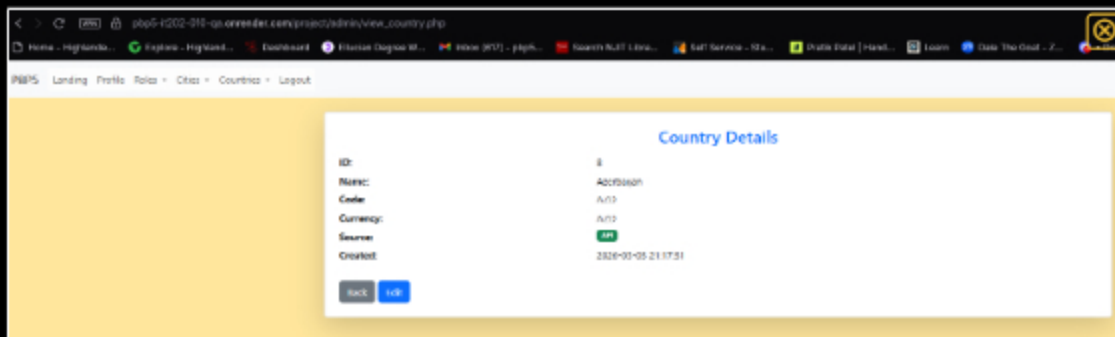




ex 2 city



ex 1 country



ex 2 country

 Saved: 5/4/2026 7:42:29 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://pbp5-it202-010-prod.onrender.com/project/admin/view_city.php?id=8



URL

<https://pbp5-it202-010-prod.c>



URL #2

<https://github.com/pbp965/pbp5-IT202-010-016>



URL

<https://github.com/pbp965/p>



URL #3

https://pbb5-it202-010-prod.onrender.com/project/admin/view_country.php?id=9



URL

<https://pbb5-it202-010-prod.c>



Saved: 5/4/2026 8:13:38 PM

Section #5: (1.5 pts.) Edit Page

Progress: 100%

≡ Task #1 (0.60 pts.) - Code

Progress: 100%

📁 Part 1:

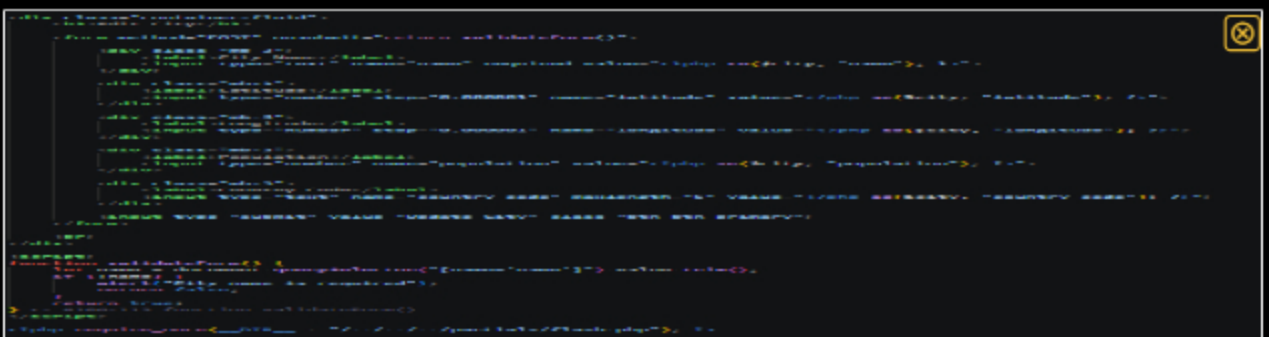
Progress: 100%

Details:

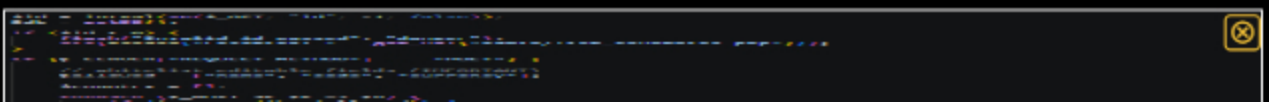
- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids
- Show the code that generates the form with the correct data types and populates it
- Show the html, javascript, and php validation (should be similar to create)
- Styling must be applied



loading of record city

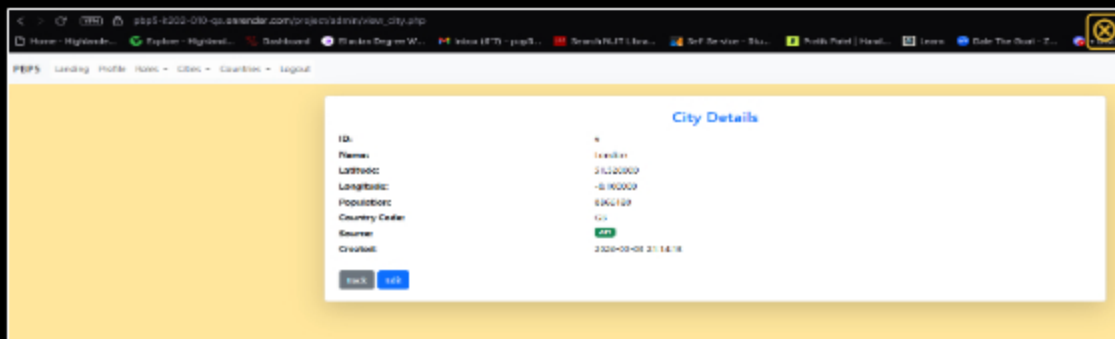


form + validations



- Show a before and after of updating an entity (clearly caption)
- Successful update should have an appropriate message shown
- The updated data should be shown in the form

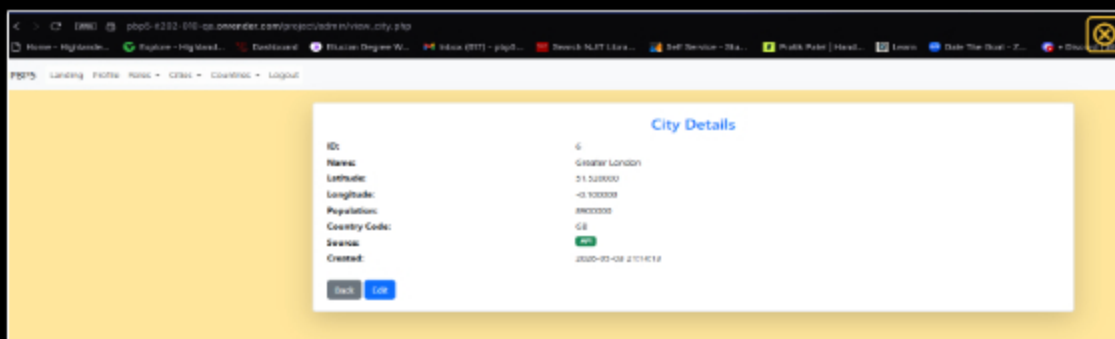
- Any errors or validation messages shown
- Show some examples of validation errors (html, javascript, php)



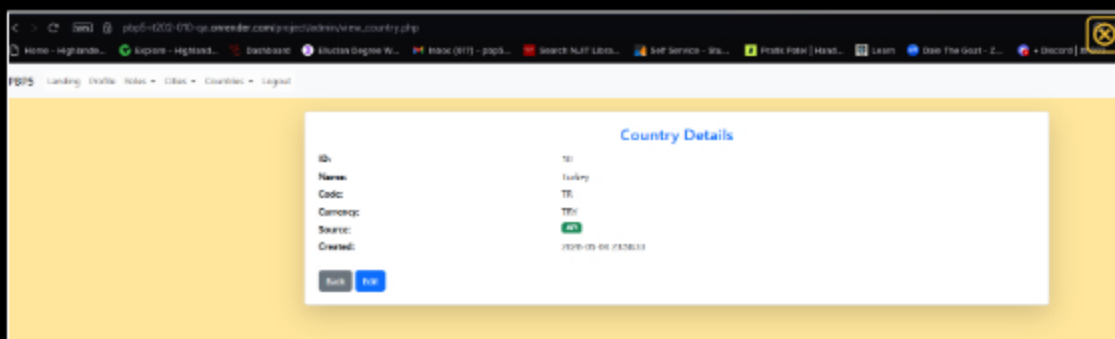
city before



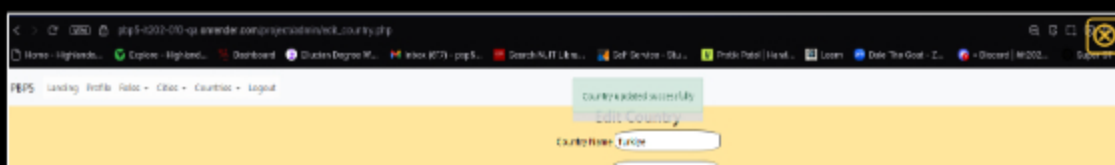
city update success



city after

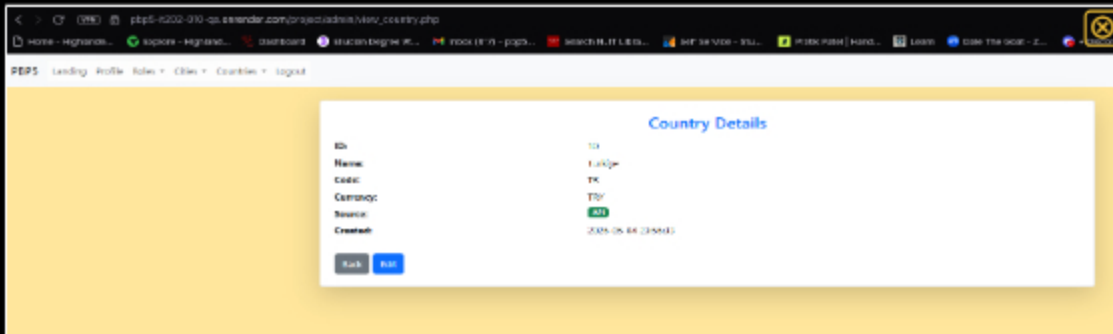


country before



Country Code: TR
Country: TR
Update Country

country update success



Country Details

ID:	10
Name:	TURKEY
Code:	TR
Country:	TR
Source:	API
Created:	2025-05-04 (23:54:03)

Back Save

country after

 Saved: 5/4/2026 7:57:35 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://pbp5-it202-010-prod.onrender.com/project/admin/view_country.php?id=10



URL

<https://pbp5-it202-010-prod.c>



URL #2

https://pbp5-it202-010-prod.onrender.com/project/admin/view_city.php?id=6



URL

<https://pbp5-it202-010-prod.c>



URL #3

<https://github.com/pbp965/pbp5-IT202-pull/44>



URL

<https://github.com/pbp965/p>



Saved: 5/4/2026 8:14:27 PM

Section #6: (1.5 pts.) Delete Logic

Progress: 100%

Task #1 (0.60 pts.) Code

Part 1:

Progress: 100%

Details:

- Show the code that handles deletion of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect to the list page with an appropriate message)
- Handle any necessary roles/permissions (i.e., it's not likely that anyone can delete any entity they choose)
- Show the redirect logic that goes back to the previous page (where the route came from), if it was a list page, it should maintain the filter/sort criteria

```
<td>
  <a href="<?php echo get_url("admin/edit_city.php"); ?>id=<?php se($row, "id"); ?>">Edit</a> |
  <a href="<?php echo get_url("admin/view_city.php"); ?>id=<?php se($row, "id"); ?>">View</a> |
  <form method="POST" style="display:inline;" onsubmit="return confirm('Delete this city?');">
    <input type="hidden" name="id" value="<?php se($row, "id"); ?>">
    <input type="hidden" name="action" value="delete">
    <button type="submit" class="btn btn-link p-0 m-0 align-baseline text-danger">
      Delete
    </button>
  </form>
</td>
```

delete city

```
<td>
  <a href="<?php echo get_url("admin/edit_country.php"); ?>id=<?php se($row, "id"); ?>">Edit</a> |
  <a href="<?php echo get_url("admin/view_country.php"); ?>id=<?php se($row, "id"); ?>">View</a> |
  <form method="POST" style="display:inline;" onsubmit="return confirm('Delete this country?');">
    <input type="hidden" name="id" value="<?php se($row, "id"); ?>">
    <input type="hidden" name="action" value="delete">
    <button type="submit" class="btn btn-link p-0 m-0 align-baseline text-danger">
      Delete
    </button>
  </form>
</td>
```

delete country



Saved: 5/4/2026 8:09:47 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the deletion logic works
- Is it a soft or hard delete?
- Note any permissions/rules applied.
- Briefly explain how the redirect maintains the previous route and any filter/sort data

Your Response:

Instance is deleted when pressing the delete button, deletion message is flashed if successful. It is a hard delete as the instance's row is removed from the db.



Saved: 5/4/2026 8:09:47 PM

Task #2 (0.60 pts.) - Examples

Progress: 100%

Details:

- Show a successful delete message
- Show a before and after database view of the record being deleted (clearly caption)
- Any errors should have user-friendly messages shown

Country List

Country Name:
Country Code:
Country ISO:
Country ID:

ID	Name	Code	Country	Source	Action
1	Turkey	TR	TUR	API	Add View Delete
2	Italy	IT	ITA	API	Add View Delete
3	Australia	AU	AUS	API	Add View Delete
4	Mexico	MX	MEX	API	Add View Delete
5	Canada	CA	CAN	Manual	Add View Delete

country before

Country List

Country Name:
Country Code:
Country ISO:
Country ID:

ID	Name	Code	Country	Source	Action
1	Turkey	TR	TUR	API	Add View Delete
2	Italy	IT	ITA	API	Add View Delete
3	Mexico	MX	MEX	API	Add View Delete
4	Canada	CA	CAN	Manual	Add View Delete

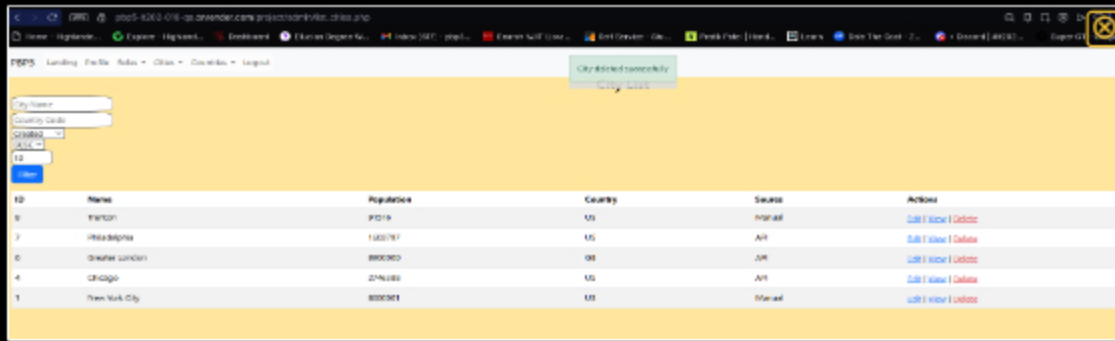
country after

City List

City Name:
Country Code:
Country ISO:
City ID:

ID	Name	Population	Country	Source	Action
1	Turkey	85,712,129	TR	Manual	Add View Delete
2	India	1,428,627,683	IN	API	Add View Delete
3	United States	331,002,651	US	API	Add View Delete
4	China	1,412,600,000	CN	API	Add View Delete
5	Germany	83,240,000	DE	API	Add View Delete
6	France	68,720,000	FR	API	Add View Delete
7	Japan	126,400,000	JP	API	Add View Delete
8	South Korea	51,700,000	KR	Manual	Add View Delete

city before



ID	Name	Population	Country	Source	Actions
6	London	91219	UK	Internal	Edit View Delete
7	Philadelphia	1583767	US	API	Edit View Delete
8	Greater London	8903002	GB	API	Edit View Delete
4	Chicago	2746288	US	API	Edit View Delete
1	New York City	8000001	US	Internal	Edit View Delete

city after



Saved: 5/4/2026 8:07:08 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page (even though it's not a full page, it's more like how logout works)
- Include pull request link for this feature

URL #1

https://pbb5-it202-010-prod.onrender.com/project/admin/list_cities.php



URL

<https://pbb5-it202-010-prod.c>



URL #2

https://pbb5-it202-010-prod.onrender.com/project/admin/list_countries.php



URL

<https://pbb5-it202-010-prod.c>



URL #3

<https://github.com/pbb965/pbb5-IT202-010-03>



URL

<https://github.com/pbb965/p>



Saved: 5/4/2026 8:12:47 PM

Section #7: (0.5 pts.) Misc

Progress: 100%

Task #1 (0.17 pts.) - Github Details

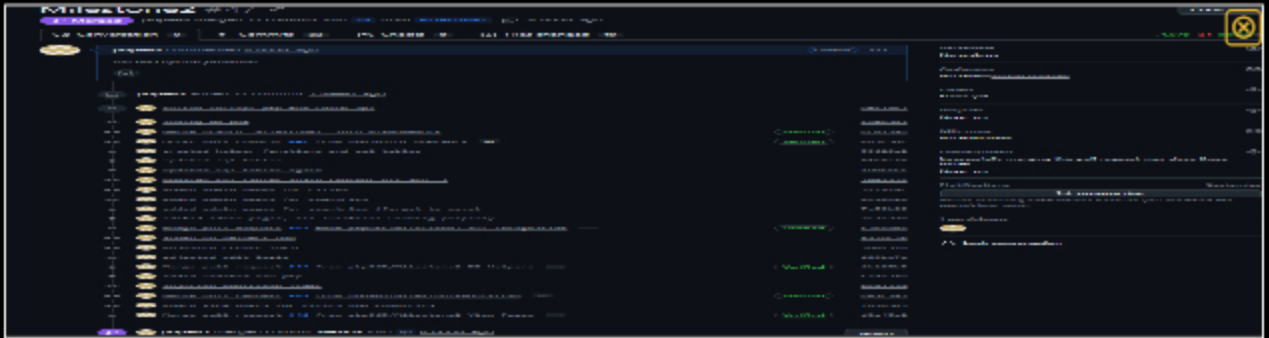
Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



commits

 Saved: 5/4/2026 8:12:30 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in /pull/#) (Milestone2 into qa)

URL #1

<https://github.com/pbp965/pbp5-IT202-010/47>



URL

<https://github.com/pbp965/pbp5-IT202-010/47>

 Saved: 5/4/2026 8:12:30 PM

Task #2 (0.17 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click Projects and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Projects • pbp5-IT202-010

timeline

Timeline	Timeline
1. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
2. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
3. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
4. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
5. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
6. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
7. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
8. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
9. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
10. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
11. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
12. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
13. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
14. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
15. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
16. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
17. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
18. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
19. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
20. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
21. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
22. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
23. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
24. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
25. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
26. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
27. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
28. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
29. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
30. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
31. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
32. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
33. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
34. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
35. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
36. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
37. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
38. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
39. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
40. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
41. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
42. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
43. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
44. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
45. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
46. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
47. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
48. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
49. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
50. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
51. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
52. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
53. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
54. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
55. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
56. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
57. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
58. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
59. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
60. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
61. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
62. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
63. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
64. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
65. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
66. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
67. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
68. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
69. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
70. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
71. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
72. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
73. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
74. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
75. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
76. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
77. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
78. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
79. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
80. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
81. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
82. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
83. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
84. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
85. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
86. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
87. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
88. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
89. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
90. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
91. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
92. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
93. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
94. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
95. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
96. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
97. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
98. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
99. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM
100. 5/4/2026 8:17:56 PM	5/4/2026 8:17:56 PM

projects



Saved: 5/4/2026 8:17:56 PM

Task #3 (0.17 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

Being able to SQL databases and an API of our choice to grab information from that API and to put it into a DB. We also learned how to display, create, list, and update that information. We also learned how to implement bootstrap into our application.



Saved: 5/4/2026 8:19:42 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of this assignment was creating the respective SQL tables and database.



Saved: 5/4/2026 8:21:59 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of this assignment was transforming our information for our DB. Another difficult part was finding an appropriate API to implement.



Saved: 5/4/2026 8:24:43 PM